

28th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2024)

On the Effects of Similarity in Community Detection

Motaz Ben Hassine^{a,b,*}, Mourad Kmimech^c, Mohamed Graiet^d^aCRIL, University of Artois and CNRS, Lens 62300, France.^bUR-OASIS-ENIT, University of Monastir, Monastir 5000, Tunisia.^cEFREI, Paris-Panthéon-Assas University, Villejuif 94800, France.^dLS2N Nantes, Nantes 44321, France.

Abstract

Community detection in social networks is a significant area of research within Artificial Intelligence and social network analysis. The agglomerative method is a well-known approach used for detecting communities. This technique relies on local similarities, to form clusters between pairs of nodes in the graph. The process involves merging each pair of nodes that exhibits the highest similarity and then computing a quality function for the current clustering results; each such merge, along with the computation of the quality of results, constitutes a step or called iteration. The choice of a similarity function can play a crucial role in determining the number of iterations, which in turn affects the running time of an agglomerative method. This raises a fundamental question when employing an agglomerative approach: a worth-asking question is how to determine which similarity function to choose and why. To address this question, our paper delves into the comparison of two well-known similarity functions: structural similarity and hub-promoted similarity. We conducted a deep comprehensive theoretical analysis followed by an extensive experimentation on several datasets, focusing on the computational aspects of these functions. Notably, our findings highlight that in specific cases within the graph, the hub-promoted similarity function is faster compared to structural similarity.

© 2024 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer-review under responsibility of the scientific committee of the 28th International Conference on Knowledge Based and Intelligent information and Engineering Systems

Keywords: Community detection; Social network; Graphs; Similarity; Clustering; Agglomerative Methods

1. Introduction

Social network analysis is an essential area within artificial intelligence, focusing on tasks such as influence maximization and community detection. Community detection, in particular, seeks to identify groups of users sharing similar interests; for instance, users interested in sports might form a distinct community. In graph theory, a social network is modeled as an undirected graph where vertices epitomize users, and edges designate the interactions or social relationships among them. An example of such a social interaction is the friend relationship on Facebook. Ana-

* Corresponding author. Tel.: +33-656690440

E-mail address: benhassine@cril.fr

lyzing social networks is essential for various applications, particularly in digital marketing. Identifying communities within these networks can significantly enhance understanding of the target audience and improve sales strategies for products or services.

Several approaches to community detection were proposed, which can be organized into two main classes: *overlapping* and *non-overlapping* methods. Overlapping techniques, such as those suggested by Shen et al. [22], Rees et al. [19], and Palla et al. [1], focused on recognizing communities where nodes may belong to multiple groups. For instance, the EAGLE algorithm by Shen et al. [22] identified maximal cliques as initial communities and then merged the most similar ones. Similarly, Li Ni et al. [15] proposed a method that detected nodes involved in multiple communities, selected representative nodes from them, and then determined the communities surrounding these representative nodes. Recent developments have introduced various overlapping community detection methods [28, 16, 24]. The second category involved non-overlapping community detection, where each node is assigned to exactly one community. Techniques in this category, such as those introduced by Zardi et al. [26] and Pasquale et al. [13], often employed modularity optimization. For example, Radicchi et al. [17] and Newman et al. [14] developed methods that either defined or optimized community structures based on quantitative definitions or quality functions like modularity. Agglomerative methods (AM, for short) represent another non-overlapping approach, relying on similarity metrics such as Salton similarity or structural similarity [21, 3], Hub-promoted similarity [18], Hub-Depressed similarity [27], α -Structural similarity [6], among others. These methods prioritize merging node pairs with the highest similarities and continue merging as long as it increases the overall modularity. The choice of the similarity function for an AM can significantly impact both the quality of clustering results and runtime. This raises an important question: what is the most suitable similarity function to use for an AM? While it is challenging to estimate or select the best link prediction function for quality purposes due to the variety of network structures and quality measures, it is feasible to determine the appropriate similarity function based on runtime considerations. This paper focuses on a comparative study of the impact of different similarity functions on the runtime of an AM. Given the variety of similarity functions available, we limited our theoretical and experimental analysis to two well-known functions: structural similarity (2S, for short) and Hub-promoted similarity (HP, for short).

The rest of the paper is structured as follows: Section 2 deals with formal notations, Section 3 presents a deep theoretical comparison between 2S and HP, Section 4 details our experiments, and Section 5 concludes the paper with hints on forthcoming work.

2. Formal notations

In this paper, we examine a simple undirected graph denoted as $G = (V, E)$, where V is the set of vertices, and E is the set of edges. The set of **neighbors** of a node $u \in V$ is defined as $N(u) = \{v \mid (u, v) \in E\}$. The **degree** of $u \in V$ is defined as the cardinality of $N(u)$, denoted as $|N(u)|$. We define the set $adj(u)$ as the union of $N(u)$ and u , i.e., $adj(u) = N(u) \cup \{u\}$. It should be noted that if $(u, v) \in E$ then $|adj(u)| \geq 2$ and $|adj(v)| \geq 2$. Additionally, let introduce the concept of **communities** in the graph $G = (V, E)$, denoted as $C_G = \{c_1, c_2, \dots, c_m\}$, where a community is a subgroup of vertices. Finally, we introduce $d_{\max}(G)$ as the **maximum degree** of the graph G , i.e., $d_{\max}(G) = \max_{|N(u)|} \{u \in V\}$.

The similarity measures 2S [21, 4] and HP [18] are local functions that rely on the immediate neighborhood between vertices. These similarity functions are based on the common neighbors between two nodes. If two nodes have many common neighbors, the value of these similarities is close to 1. Otherwise, the value is close to 0. These metrics are formally defined below:

Definition 1 (Structural Similarity). *Let $G = (V, E)$ be an undirected graph and $(u, v) \in E$, then the 2S of u and v , denoted by $s_2(u, v)$, is defined as:*

$$s_2(u, v) = \frac{|adj(u) \cap adj(v)|}{\sqrt{|adj(u)| \times |adj(v)|}} \quad (1)$$

It is important to note that $s_2(u, v) \in [0..1]$.

Definition 2 (Ravasz Hub-Promoted Similarity). Let $G = (V, E)$ be an undirected graph and $(u, v) \in E$. Then, the HP of u and v , denoted by $s_{RHP}(u, v)$, is defined as:

$$s_{RHP}(u, v) = \begin{cases} \frac{|N(u) \cap N(v)|}{\min(|N(u)|, |N(v)|)}, & \text{if } N(u) \cap N(v) \neq \emptyset \\ \frac{1}{\min(|N(u)|, |N(v)|)}, & \text{if } N(u) \cap N(v) = \emptyset \end{cases} \quad (2)$$

Let us define the quality of the set of communities C_G called modularity [4], as presented in the following definition.

Definition 3 (Modularity). Let $G = (V, E)$ be an undirected graph with $C_G = \{c_1, c_2, \dots, c_m\}$ is the set of communities of G and let s be a general similarity measure, which can be either HP or 2S. The modularity is defined as:

$$Q(C_G) = \sum_{i=1}^m \left[\frac{IS_i}{TS} - \left(\frac{DS_i}{TS} \right)^2 \right] \quad (3)$$

$$\text{where } IS_i = \sum_{u, v \in c_i} s(u, v), \quad DS_i = \sum_{u \in c_i, v \in V} s(u, v), \quad \text{and } TS = \sum_{u, v \in V} s(u, v).$$

3. Comparative Study: Structural Similarity vs. Hub-Promoted Similarity

In this section, we first aim to simplify Equation 2. Then, the formal definition of the latter is presented as follows:

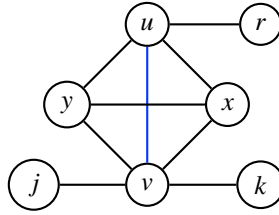
Definition 4 (Hub-Promoted similarity). Let $G = (V, E)$ be an undirected graph and $(u, v) \in E$. Then, the HP of u and v , denoted by $s_{hp}(u, v)$, is defined as:

$$s_{hp}(u, v) = \frac{|adj(u) \cap adj(v)|}{\min(|adj(u)|, |adj(v)|)} \quad (4)$$

It is important to note that $s_{hp}(u, v) \in]0..1]$.

Then, we illustrate the behaviour of the simplified HP through the following example.

Example 1. Let us consider the undirected graph depicted in Figure 1.



Then, we have that:

$$\begin{aligned}
 s_{hp}(u, v) &= \frac{|adj(u) \cap adj(v)|}{\min(|adj(u)|, |adj(v)|)} \\
 &= \frac{|{u, y, v, x, r} \cap {v, y, x, u, j, k}|}{\min(|{u, y, v, x, r}|, |{v, y, x, u, j, k}|)} \\
 &= \frac{|{u, y, v, x}|}{\min(|{u, y, v, x, r}|, |{v, y, x, u, j, k}|)} \\
 &= \frac{4}{\min(5, 6)} = \frac{4}{5} = 0.8
 \end{aligned}$$

Fig. 1. Example of computing the HP for the edge (u, v) in a simple undirected graph.

In what follows, we aim to theoretically determine whether an AM using the HP measure is faster than using the 2S measure or not. In the process of finding communities, a set of edges is selected and merged, followed by the computation of modularity. These actions constitute an **iteration**. This iteration is repeated to achieve the optimal result. Then, our goal is to compare the number of iterations required by 2S and HP. Before delving into the comparison, we propose several formal definitions.

Definition 5 (Similar edges). Let $G = (V, E)$ be an undirected graph and $(u, v), (u', v') \in E$. Then, (u, v) and (u', v') are called **similar edges** if:

- $|adj(u)| = |adj(u')|$ and $|adj(v)| = |adj(v')|$ and $|adj(u) \cap adj(v)| = |adj(u') \cap adj(v')|$, or
- $|adj(u)| = |adj(v)| = |adj(u) \cap adj(v)|$ and $|adj(u')| = |adj(v')| = |adj(u') \cap adj(v')|$

Definition 6 (Pseudo-similar edges). Let $G = (V, E)$ be an undirected graph and $(u, v), (u', v') \in E$. Then, (u, v) and (u', v') are called **pseudo-similar edges** if:

- $|adj(u)| = |adj(u')|$ and $|adj(v)| \neq |adj(v')|$ and,
- $(|adj(u)| = |adj(v)|$ and $|adj(u')| < |adj(v')|$ or $|adj(u)| < |adj(v)|$ and $|adj(u')| < |adj(v')|)$ and,
- $|adj(u) \cap adj(v)| = |adj(u') \cap adj(v')|$

In the sequel, our goal is to analyze whether the similarities of (u, v) and (u', v') are equal or different for each of Definitions 5 and 6. We present the following propositions:

Proposition 1. Let $G = (V, E)$ be an undirected graph. Let $(u, v), (u', v') \in E$. If (u, v) and (u', v') are similar edges, then $s_2(u, v) = s_2(u', v')$ and $s_{hp}(u, v) = s_{hp}(u', v')$.

Proof. (See [Appendix A](#)) □

Proposition 2. Let $G = (V, E)$ be an undirected graph and $(u, v), (u', v') \in E$. If (u, v) and (u', v') are pseudo-similar edges, then $s_2(u, v) \neq s_2(u', v')$ and $s_{hp}(u, v) = s_{hp}(u', v')$.

Proof. (See [Appendix B](#)) □

Proposition 1 states that HP and 2S exhibit the same behavior in the case of similar edges. However, proposition 2 shows that 2S and HP are different. Specifically, in the case of pseudo-similar edges, the 2S of (u, v) and (u', v')

are different, whereas, for the HP of (u, v) and (u', v') is the same. As a result, HP minimizes the difference in edge values, leading to a reduction in the number of iterations required. Afterwards, our objective is to demonstrate that the number of iterations using HP is lower than the number of iterations using 2S. Prior to delving into the comparison, we propose with the following four definitions.

Definition 7 (Similar Edges Set-Based 2S). Let $G = (V, E)$ be an undirected graph and $Z^{s_2} \subseteq E$. Then, Z^{s_2} is called **Similar Edges Set-Based 2S** if $\forall (u, v), (u', v') \in Z^{s_2}$, we have $s_2(u, v) = s_2(u', v')$.

Definition 8 (Similar Edges Set-Based HP). Let $G = (V, E)$ be an undirected graph and $Z^{s_{hp}} \subseteq E$. Then, $Z^{s_{hp}}$ is called **Similar Edges Set-Based HP** if $\forall (u, v), (u', v') \in Z^{s_{hp}}$, we have $s_{hp}(u, v) = s_{hp}(u', v')$.

Definition 9 (Pseudo-similar edges set). Let $G = (V, E)$ be an undirected graph. The **set of pseudo-similar edges** is defined as $A = \{(u, v), (u', v') \mid (u, v), (u', v') \in E \text{ and } (u, v), (u', v') \text{ are pseudo-similar edges}\}$

Definition 10 (Other edges set). Let $G = (V, E)$ be an undirected graph. The **set of Other edges** is defined as $B = \{(u, v), (u', v') \mid (u, v), (u', v') \in E \text{ and } (u, v), (u', v') \text{ are not pseudo-similar edges}\}$

Conjecture 1. Let $G = (V, E)$ be an undirected graph, where $E = \bigcup_{i=1}^{P_{s_2}} Z_i^{s_2}$ and $E = \bigcup_{j=1}^{P_{s_{hp}}} Z_j^{s_{hp}}$. If $|A| \geq |B|$, then $P_{s_{hp}} \leq P_{s_2}$.

Conjecture 1 states that if there are many pairs of edges in a given undirected graph that are pseudo-similar to each other, then the number of iterations when using HP is lower or equal to the number of iterations when using 2S. This implies that employing HP in AM results in faster convergence than using 2S.

Example 2. Let us consider an undirected graph depicted in Figure 2. Let us suppose that the final clustering result is $C_G = \{\{1, 2, 3\}, \{4, 5, 6, 7\}\}$. This figure illustrates the distribution of similarity values in the graph. As shown, when using 2S, we have 4 similar edges sets ($P_{s_2} = 4$), indicating that the number of iterations is 4. However, when using HP, we only have 3 similar edges sets ($P_{s_{hp}} = 3$), reducing the number of iterations to 3. This reduction occurs because there are some edges like $(1, 3)$ (colored in blue) and $(1, 2)$ (colored in red) which are pseudo-similar. According to HP, these edges must have the same similarity value, which further reduces the number of iterations required.

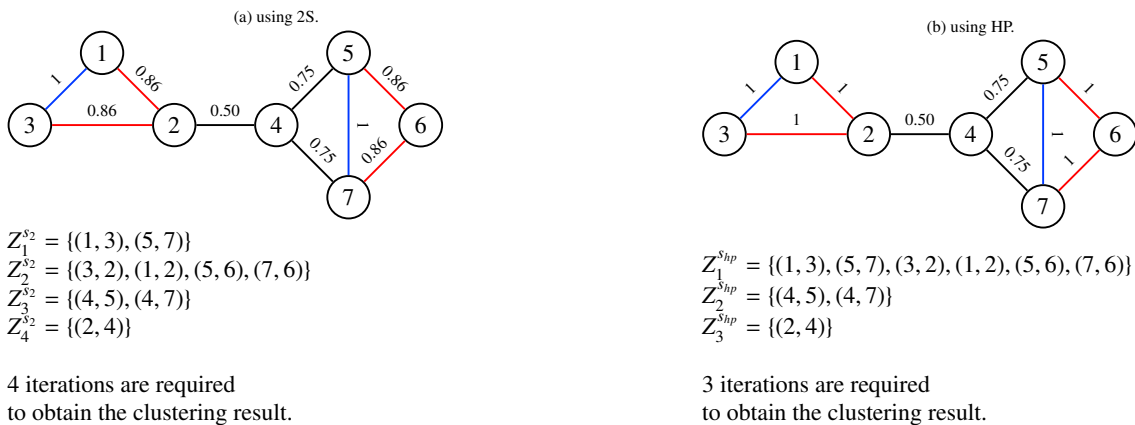


Fig. 2. An example of comparing the iteration counts between 2S and HP.

Next, let us explain the steps of the AM that we adopted:

1. Compute the similarity for each edge in graph G using 2S or HP.
2. Initialize the set of communities C_G , with each vertex initially considered as an individual community.
3. Compute the modularity $Q(C_G)$ corresponding to this initial clustering configuration.
4. In each iteration, merge the pair of nodes that exhibit the strongest similarity among all pairs in the graph.

5. Recalculate the modularity $Q(C_G)$ for the current set of merged communities.
6. If the modularity $Q(C_G)$ increases after the merging step, continue the process and return to step 4. Otherwise, halt the merging process. At this stage, consider the previous result as the best clustering set achieved.

Complexity analysis: Typically, in each iteration, we compute the modularity and merge a pair of nodes, resulting in a time complexity of $O(|E| + |V|)$ [11]. In conjecture 1, AM using 2S requires P_{s_2} iterations, while AM using HP requires $P_{s_{hp}}$ iterations. The computation of 2S and HP has a time complexity of $O(|V|d_{\max}(G)^3)$ [12]. Therefore, the total complexity of AM using 2S is $O(|V|d_{\max}(G)^3 + (|E| + |V|)P_{s_2})$, and the total complexity of AM using HP is $O(|V|d_{\max}(G)^3 + (|E| + |V|)P_{s_{hp}})$. Since we have $P_{s_{hp}} \leq P_{s_2}$ (as stated in conjecture 1), we can deduce that $O(|V|d_{\max}(G)^3 + (|E| + |V|)P_{s_{hp}}) \leq O(|V|d_{\max}(G)^3 + (|E| + |V|)P_{s_2})$. Therefore, AM using HP is faster than AM using 2S. In summary, HP outperforms 2S due to the reduced number of iterations required.

4. Experiments

To prove our analysis, we put forward an implementation, our code and the datasets are uploaded on GitHub repository [5]. We conducted two kinds of experiments. Firstly, we evaluated the performance of AM using HP (AM-HP, for short) and AM using 2S (AM-2S, for short) on artificial networks called synthetic networks [8]. We varied a parameter called the mixing parameter μ within the range of 0.1 to 0.9. This parameter plays a crucial role in controlling the network's structure. Our goal is to confirm our theoretical analysis, by demonstrating that on many different network structures, the number of iterations required for AM-HP can be less than or equal to the number of iterations for AM-2S. Consequently, we expect the running time of AM-HP to be lower than AM-2S. For each value of μ , we compare AM-2S and AM-HP based on the number of iterations and the running time. Secondly, we further compare AM-2S and AM-HP on 8 renowned real-world datasets, analyzing the number of iterations and running time. Our experiments are conducted on a computer with 12 GB of memory, 500 GB of storage, and CPU i7. To set up our implementation, we utilized an open-source scientific environment named Spyder installed on Windows 10. We presented all the datasets in Tables 1 and 2.

Table 1. Real networks.

Datasets	Nodes/Edges	Source
Karate	34/78	[25]
Dolphin	62/159	[10]
Books	105/441	[7]
Citeseer	3264/4536	[2]
Email-Eu-Core	1005/25571	[9]
Cora	23166/89157	[23]
Amazon	334863/925872	[9]
YouTube	1134890/2987624	[9]

Table 2. Synthetic networks.

μ	Nodes	Source
[0.1 ... 0.9]	1000	[20]
[0.1 ... 0.9]	5000	[20]
[0.1 ... 0.9]	10000	[20]
[0.1 ... 0.9]	50000	[20]

In the initial phase of the experiment, we compared the performance of AM-HP to AM-2S in terms of the number of iterations and running time. This comparison is conducted on various synthetic networks. The results are illustrated in Figures 3, 4, 5, 6, 7, 8, 9, and 10. The bar charts illustrate that regardless of the variation

in network structures, the number of iterations required for AM-HP is always lower than AM-2S. These findings confirm the results of conjecture 1. Additionally, the bar charts reveal that AM-HP is faster than AM-2S.

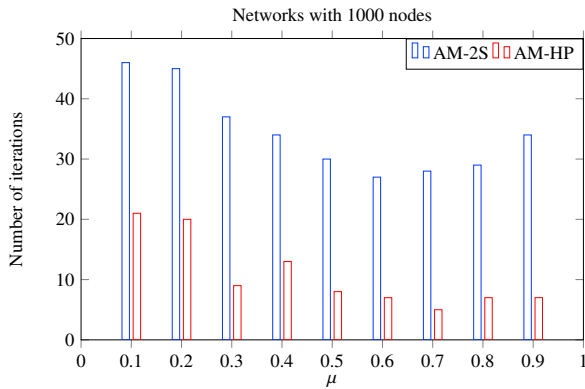


Fig. 3. AM-2S vs. AM-HP on synthetic networks with 1000 nodes based on the number of iterations.

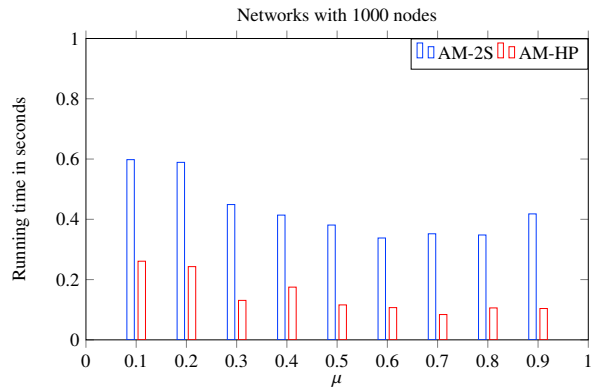


Fig. 4. AM-2S vs. AM-HP on synthetic networks with 1000 nodes based on the running time.

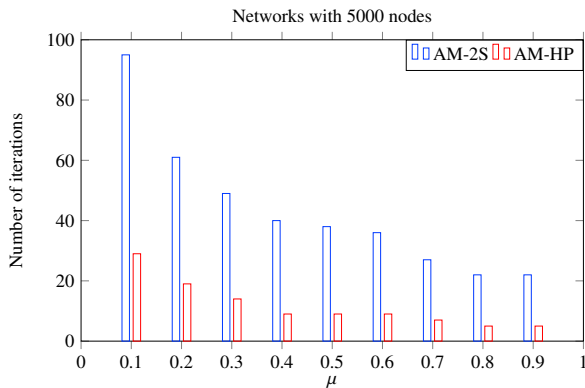


Fig. 5. AM-2S vs. AM-HP on synthetic networks with 5000 nodes based on the number of iterations.

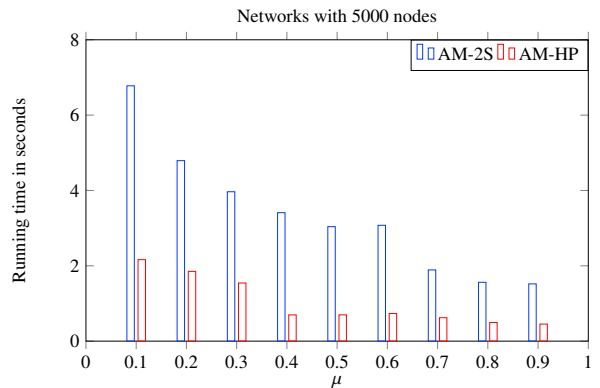


Fig. 6. AM-2S vs. AM-HP on synthetic networks with 5000 nodes based on the running time.

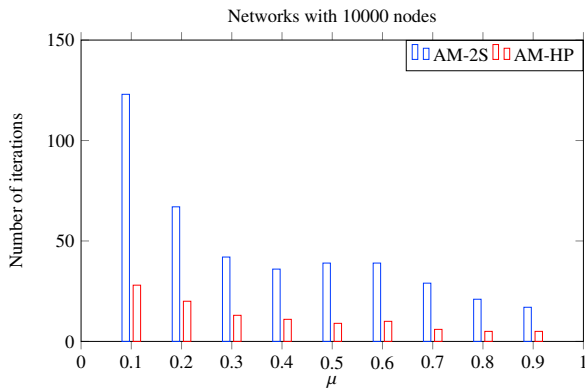


Fig. 7. AM-2S vs. AM-HP on synthetic networks with 10000 nodes based on the number of iterations.

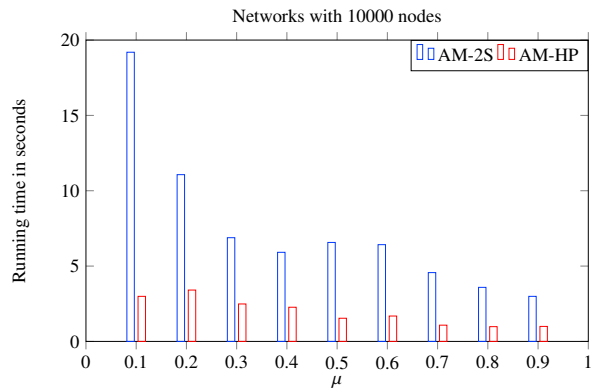


Fig. 8. AM-2S vs. AM-HP on synthetic networks with 10000 nodes based on the running time.

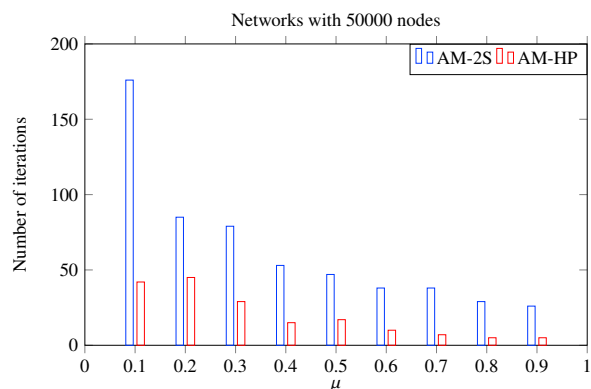


Fig. 9. AM-2S vs. AM-HP on synthetic networks with 50000 nodes based on the number of iterations.

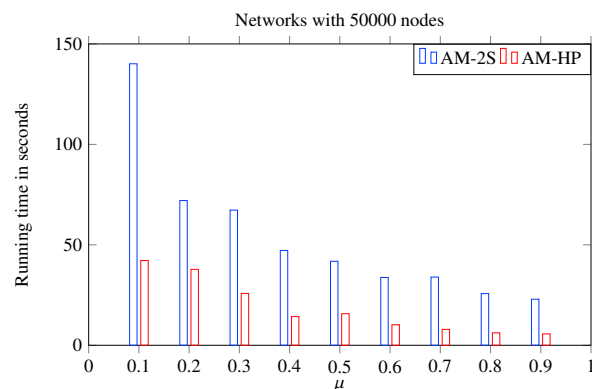


Fig. 10. AM-2S vs. AM-HP on synthetic networks with 50000 nodes based on the running time.

In the second stage of the experiment, we compared AM-HP to AM-2S on 8 real-world datasets, in terms of running time and the number of iterations. As shown in Table 3, the number of iterations required for AM-HP is lower than that of AM-2S across all real-world datasets. This outcome further validates the results of conjecture 1. Additionally, Table 3 indicates that AM-HP exhibits faster performance compared to AM-2S.

Table 3. AM-HP vs. AM-2S based on the number of iterations and running time.

Datasets	AM-2S		AM-HP	
	Running time(s)	Iterations	Running time (s)	Iterations
Karate	0.011	27	0.003	8
Email-Eu-Core	11.173	230	1.676	37
Citeseer	7.41	192	0.967	22
Dolphin	0.033	42	0.01	13
Books	0.143	103	0.023	16
Amazon	8611.44	1043	1143.428	197
Cora	428.89	708	26.057	53
YouTube	5965.827	253	397.165	5

5. Conclusion and forthcoming work

In this paper, we presented a deep comparative study of two well-known similarity functions, 2S and HP, to explore their impact on the computational efficiency of the agglomerative clustering method. We provided a theoretical analysis indicating that for a given undirected graph where exists many pairs of edges that are pseudo-similar, using HP requires a lower number of iterations compared to 2S. Therefore, using HP for an AM is faster than using 2S. To substantiate our theoretical predictions, we performed extensive empirical tests across various datasets. The experimental results confirmed that HP requires a lower number of iterations compared to 2S. Then, AM-HP is faster than AM-2S. In the future, we will expand this analysis to include additional similarity functions, aiming to broaden the applicability of our findings and better understand how similarity measures influence the performance of clustering algorithms.

References

- [1] Ahn, Y.Y., Bagrow, J.P., Lehmann, S., 2010. Link communities reveal multiscale complexity in networks. *nature* 466, 761–764.
- [2] Bollacker, K.D., Lawrence, S., Giles, C.L., 1998. Citeseer: An autonomous web agent for automatic retrieval and identification of interesting publications, in: *Proceedings of the second international conference on Autonomous agents*, pp. 116–123.

- [3] Chen, Y., Zhu, W., Peng, W., Lee, W., Lee, S., 2014a. CIM: community-based influence maximization in social networks. *ACM Trans. Intell. Syst. Technol.* 5, 25:1–25:31. URL: <https://doi.org/10.1145/2532549>, doi:10.1145/2532549.
- [4] Chen, Y.C., Zhu, W.Y., Peng, W.C., Lee, W.C., Lee, S.Y., 2014b. Cim: community-based influence maximization in social networks. *ACM Transactions on Intelligent Systems and Technology (TIST)* 5, 1–31.
- [5] Hassine, M.B., 2024. Comparative study project. URL: <https://github.com/2x254/Comparative-Study-Implementation.git>.
- [6] Hassine, M.B., Jabbour, S., Kmimech, M., Raddaoui, B., Graiet, M., 2023. A non-overlapping community detection approach based on α -structural similarity, in: Wrembel, R., Gamper, J., Kotsis, G., Tjoa, A.M., Khalil, I. (Eds.), *Big Data Analytics and Knowledge Discovery - 25th International Conference, DaWaK 2023, Penang, Malaysia, August 28-30, 2023, Proceedings*, Springer. pp. 197–211. URL: https://doi.org/10.1007/978-3-031-39831-5_19, doi:10.1007/978-3-031-39831-5_19.
- [7] Krebs, V., 2004. Books about us politics. unpublished, <http://www.orgnet.com>.
- [8] Lancichinetti, A., Fortunato, S., Radicchi, F., 2008. Benchmark graphs for testing community detection algorithms. *Physical Review E* 78, doi:10.1103/physreve.78.046110.
- [9] Leskovec, J., Krevl, A., 2014. Snap datasets: Stanford large network dataset collection at <http://snap.stanford.edu/data>.
- [10] Lusseau, D., Schneider, K., Boisseau, O.J., Haase, P., Sooten, E., Dawson, S.M., 2003. The bottlenose dolphin community of doubtful sound features a large proportion of long-lasting associations. *Behavioral Ecology and Sociobiology* 54.
- [11] Lufs Rita, 2020. Towards data science: Modularity maximization greedy algorithm URL: <https://towardsdatascience.com/modularity-maximization-5cfa6495b286>.
- [12] Martínez, V., Berzal, F., Cubero, J.C., 2016. A survey of link prediction in complex networks. *ACM Comput. Surv.* 49. URL: <https://doi.org/10.1145/3012704>.
- [13] Meo, P.D., Ferrara, E., Fiumara, G., Provetti, A., 2011. Generalized louvain method for community detection in large networks. *CoRR abs/1108.1502*. URL: <http://arxiv.org/abs/1108.1502>, arXiv:1108.1502.
- [14] Newman, M.E., 2006. Modularity and community structure in networks. *Proceedings of the national academy of sciences* 103, 8577–8582.
- [15] Ni, L., Luo, W., Zhu, W., Hua, B., 2020. Local overlapping community detection. *ACM Trans. Knowl. Discov. Data* 14, 3:1–3:25. URL: <https://doi.org/10.1145/3361739>, doi:10.1145/3361739.
- [16] Niu, Y., Kong, D., Liu, L., Wen, R., Xiao, J., 2023. Overlapping community detection with adaptive density peaks clustering and iterative partition strategy. *Expert Systems with Applications* 213, 119213.
- [17] Radicchi, F., Castellano, C., Cecconi, F., Loreto, V., Parisi, D., 2004. Defining and identifying communities in networks. *Proceedings of the national academy of sciences* 101, 2658–2663.
- [18] Ravasz, E., Somera, A.L., Mongru, D.A., Oltvai, Z.N., Barabási, A.L., 2002. Hierarchical organization of modularity in metabolic networks. *science* 297, 1551–1555.
- [19] Rees, B.S., Gallagher, K.B., 2010. Overlapping community detection by collective friendship group inference, in: Memon, N., Alhajj, R. (Eds.), *International Conference on Advances in Social Networks Analysis and Mining, ASONAM 2010, Odense, Denmark, August 9-11, 2010*, IEEE Computer Society. pp. 375–379.
- [20] Rossetti, G., Milli, L., Cazabet, R., 2019. Cdlib: a python library to extract, compare and evaluate communities from complex networks. *Applied Network Science*.
- [21] Salton, G., 1983. *Introduction to modern information retrieval*. McGraw-Hill.
- [22] Shen, H., Cheng, X., Cai, K., Hu, M.B., 2009. Detect overlapping and hierarchical community structure in networks. *Physica A: Statistical Mechanics and its Applications* 388, 1706–1712.
- [23] Šubelj, L., Bajec, M., 2013. Model of complex networks based on citation dynamics, in: *Proceedings of the 22nd international conference on World Wide Web*.
- [24] Yuan, S., Zeng, H., Zuo, Z., Wang, C., 2023. Overlapping community detection on complex networks with graph convolutional networks. *Computer Communications* 199, 62–71.
- [25] Zachary, W.W., 1977. An information flow model for conflict and fission in small groups. *Journal of anthropological research* 33.
- [26] Zardi, H., Romdhane, L.B., 2013. An $O(n^2)$ algorithm for detecting communities of unbalanced sizes in large scale social networks. *Knowl. Based Syst.* 37, 19–36. URL: <https://doi.org/10.1016/j.knosys.2012.05.021>, doi:10.1016/J.KNOSYS.2012.05.021.
- [27] Zhou, T., Lü, L., Zhang, Y.C., 2009. Predicting missing links via local information. *The European Physical Journal B* 71, 623–630.
- [28] Zhuo, Z., Chen, B., Yu, S., Cao, L., 2024. Overlapping community detection using expansion with contraction. *Neurocomputing* 565, 126989.

Appendix A.

- if $|adj(u)| = |adj(u')|$ and $|adj(v)| = |adj(v')|$ and $|adj(u) \cap adj(v)| = |adj(u') \cap adj(v')|$

– According to 2S :

$$\begin{aligned} \text{We have } |adj(u)| \times |adj(v)| &= |adj(u')| \times |adj(v')| \implies \frac{1}{\sqrt{|adj(u)| \times |adj(v)|}} = \frac{1}{\sqrt{|adj(u')| \times |adj(v')|}} \implies \\ \frac{|adj(u) \cap adj(v)|}{\sqrt{|adj(u)| \times |adj(v)|}} &= \frac{|adj(u') \cap adj(v')|}{\sqrt{|adj(u')| \times |adj(v')|}} \Leftrightarrow s_2(u, v) = s_2(u', v') \end{aligned}$$

– According to HP :

$$\begin{aligned} \text{We have } \min(|adj(u)|, |adj(v)|) &= \min(|adj(u')|, |adj(v')|) \implies \frac{1}{\min(|adj(u)|, |adj(v)|)} = \frac{1}{\min(|adj(u')|, |adj(v')|)} \implies \\ \frac{|adj(u) \cap adj(v)|}{\min(|adj(u)|, |adj(v)|)} &= \frac{|adj(u') \cap adj(v')|}{\min(|adj(u')|, |adj(v')|)} \Leftrightarrow s_{hp}(u, v) = s_{hp}(u', v') \end{aligned}$$

- if $|adj(u)| = |adj(v)| = |adj(u) \cap adj(v)|$ and $|adj(u')| = |adj(v')| = |adj(u') \cap adj(v')|$
 - According to 2S :
We have $s_2(u, v) = \frac{|adj(u) \cap adj(v)|}{\sqrt{|adj(u)| \times |adj(v)|}} = \frac{|adj(u) \cap adj(u)|}{\sqrt{|adj(u)| \times |adj(u)|}} = 1$ and we have $s_2(u', v') = \frac{|adj(u') \cap adj(v')|}{\sqrt{|adj(u')| \times |adj(v')|}} = \frac{|adj(u') \cap adj(u')|}{\sqrt{|adj(u')| \times |adj(u')|}} = 1 \Leftrightarrow s_2(u, v) = s_2(u', v')$
 - According to HP :
We have $s_{hp}(u, v) = \frac{|adj(u) \cap adj(v)|}{\min(|adj(u)|, |adj(v)|)} = \frac{|adj(u) \cap adj(u)|}{\min(|adj(u)|, |adj(u)|)} = 1$ and we have $s_2(u', v') = \frac{|adj(u') \cap adj(v')|}{\min(|adj(u')|, |adj(v')|)} = \frac{|adj(u') \cap adj(u')|}{\min(|adj(u')|, |adj(u')|)} = 1 \Leftrightarrow s_{hp}(u, v) = s_{hp}(u', v')$

Appendix B.

- if $|adj(u)| = |adj(u')|$ and $|adj(v)| \neq |adj(v')|$ and $|adj(u)| < |adj(v)|$ and $|adj(u')| < |adj(v')|$ and $|adj(u) \cap adj(v)| = |adj(u') \cap adj(v')|$
 - According to 2S :
K.T. $|adj(u)| \geq 2, |adj(v)| \geq 2, |adj(u')| \geq 2, |adj(v')| \geq 2, |adj(u) \cap adj(v)| \geq 2, |adj(u') \cap adj(v')| \geq 2$.
We have $|adj(v)| \neq |adj(v')| \Rightarrow |adj(u)| \times |adj(v)| \neq |adj(u')| \times |adj(v')| \Rightarrow \frac{1}{\sqrt{|adj(u)| \times |adj(v)|}} \neq \frac{1}{\sqrt{|adj(u')| \times |adj(v')|}} \Rightarrow s_2(u, v) \neq s_2(u', v')$
 - According to HP :
K.T. $|adj(u)| < |adj(v)|$. We have $s_{hp}(u, v) = \frac{|adj(u) \cap adj(v)|}{\min(|adj(u)|, |adj(v)|)} = \frac{|adj(u) \cap adj(v)|}{|adj(u)|} = \frac{|adj(u) \cap adj(v')|}{|adj(u')|} = \frac{|adj(u') \cap adj(v')|}{\min(|adj(u')|, |adj(v')|)} \Leftrightarrow s_{hp}(u, v) = s_{hp}(u', v')$
- if $|adj(u)| = |adj(u')|$ and $|adj(v)| \neq |adj(v')|$ and $|adj(u)| = |adj(v)|$ and $|adj(u')| < |adj(v')|$ and $|adj(u) \cap adj(v)| = |adj(u') \cap adj(v')|$
 - According to 2S :
We have $s_2(u, v) = \frac{|adj(u) \cap adj(v)|}{\sqrt{|adj(u)| \times |adj(v)|}} = \frac{|adj(u) \cap adj(u)|}{\sqrt{|adj(u)| \times |adj(u)|}} = 1$
K.T. $|adj(u')| \geq 2, |adj(v')| \geq 2$ and $|adj(u') \cap adj(v')| \geq 2$ We have $|adj(u') \cap adj(v')| \leq |adj(u')| < |adj(v')|$
 $\Rightarrow |adj(u') \cap adj(v')| < |adj(v')| \Rightarrow |adj(u') \cap adj(v')|^2 < |adj(u')| \times |adj(v')| \Rightarrow |adj(u') \cap adj(v')| < \sqrt{|adj(u')| \times |adj(v')|} \Rightarrow \frac{|adj(u') \cap adj(v')|}{\sqrt{|adj(u')| \times |adj(v')|}} < 1 \Leftrightarrow s_2(u', v') < 1 \Rightarrow s_2(u, v) \neq s_2(u', v')$
 - According to HP :
We have $s_{hp}(u, v) = \frac{|adj(u) \cap adj(v)|}{\min(|adj(u)|, |adj(v)|)} = \frac{|adj(u) \cap adj(u)|}{\min(|adj(u)|, |adj(u)|)} = 1$
 $s_{hp}(u', v') = \frac{|adj(u') \cap adj(v')|}{\min(|adj(u')|, |adj(v')|)} = \frac{|adj(u') \cap adj(v')|}{|adj(u')|} = \frac{|adj(u) \cap adj(v)|}{|adj(u)|} = \frac{|adj(u) \cap adj(u)|}{|adj(u)|} = 1 \Leftrightarrow s_{hp}(u', v') = 1 \Leftrightarrow s_{hp}(u, v) = s_{hp}(u', v')$